

Extreme Programming Compared with Microsoft-Style Iterative Development

Highlighting the similarities—although they might be superficial in some cases—in software development practices.

One of the most talked-about methodologies among the software engineering community over the past decade has been a set of principles and practices called “Extreme Programming,” or “XP.”¹ Kent Beck led in founding this movement around 1996. As I learned more about XP, it occurred to me that many of the practices were similar to iterative software engineering and prototype-driven product development in general. I am particularly familiar with the “Synchronize-and-Stabilize” techniques used at Microsoft in past years—at least before Windows became the mammoth project it is today—as well as at Netscape and other PC and Internet software companies.² In this column, I present a brief overview of a dozen key XP practices and how

they compare to a Microsoft-style iterative development process.³

²My discussion of Microsoft’s and other iterative practices such as at Netscape can be found in chapter 4 of *The Business of Software*, Free Press/Simon and Schuster, New York, 2004. See also Michael A. Cusumano and Richard W. Selby, *Microsoft Secrets*, Free Press/Simon and Schuster, New York, 1995 and Michael A. Cusumano and David B. Yoffie, *Competing on Internet Time: Lessons from Netscape and Its Battle with Microsoft*, Free Press/Simon and Schuster, New York, 1998.

¹For the basic text on XP, see Kent Beck with Cynthia Andres, *Extreme Programming Explained, Second Edition*, Addison-Wesley, Boston, 2005.

Planning Around User Stories. The goal in XP projects is to plan the project work based on three techniques: Create “user stories” that capture the functionality or features the customer wants; generate estimates from the developers of how long it will take to create each of these features (based on their prior experience); and then ask the customer to prioritize which features they want in which order, as well as how often they would like to see a working version of the product. The Microsoft-style analogy for this practice is to structure product requirements around “features” that represent small chunks of functionality understandable to users as well as to rely heavily on observations of how users actually use its products.

Small Releases of Functionality. XP projects have the goal of

³My thanks to Kent Beck for reviewing a draft of this column as well as for the many discussions we have had over the past several years about XP practices, how they compare to Microsoft development practices, and to what extent they could be useful for people interested in product development more generally.

Technology Strategy and Management

IN GENERAL, MICROSOFT PROJECTS, LIKE XP PROJECTS, CAN RE-ARCHITECT WITHIN LIMITS TO ACCOMMODATE NEW FUNCTIONALITY.

building the product around these user stories in small increments of functionality and then showing or “releasing” the evolving product to the customer or representative users as often as possible. The Microsoft-style analogy would be to evolve features in increments through “daily builds,” use these builds internally, and show the evolving product to users in usability labs, before moving to a beta release process.

System or Project Metaphor.

XP teams try to assign an organizing metaphor to each project to give everyone a common focus and to help the team and the customer understand what the project is about and what is most important. The Microsoft-style analogy is to create a short “vision statement” for each project defining the key features that will be (and not be) in the product.

Simple Design. XP team members focus on building only what user stories suggest the customer needs “today” in the simplest way to get the job done. The assumption is that future requirements will change unpredictably and therefore doing too much “design for the future” will inevitably involve too much waste of time and manpower. This is somewhat of a controversial point for large, complex software systems, but this is the idea. There is no specific

Microsoft-style analogy for this point. Early Microsoft projects (like the first version of Basic for the Altair PC) had very simple designs. But ad hoc coding and incremental feature creep, as with Windows Longhorn/Vista, has led to spaghetti-style architectures and products that are unnecessarily large and cumbersome. (See my July 2006 column “What Road Ahead for Microsoft and Windows?”, and my February 2007 column “What Road Ahead for Microsoft the Company?”.) Microsoft’s large development teams also tend to build too many features that customers never or rarely use.

Continuous Testing (or Test-Driven Design). XP project members are supposed to create two types of tests before or as they write code in order to find and fix defects as they occur as well as feel an immediate sense of progress. First, they write automated unit tests that cover small chunks of incrementally added functionality. Second, they create customer acceptance tests that check the broader functionality of features as defined in the user stories. Microsoft projects also generally require programmers and their tester “buddies” to run “smoke tests” once or more a day on newly written code before and after attempts to integrate the new

code with the project build. In the early days of the Internet, when Microsoft very rapidly introduced products such as Internet Explorer and NetMeeting, Microsoft teams also followed more of an XP style in that they sometimes had program managers skip the stage of writing functional specifications and instead focus on writing test cases while developers wrote code.⁴

Refactoring. XP programmers continue to revise their product designs and the underlying “architecture” (what subsystems exist and what interfaces they have to other subsystems or systems) to accommodate new functionality as well as to eliminate redundant code. This “refactoring” occurs in Microsoft-style projects as well. For example, the Office and Windows teams incrementally re-architected during the 1990s and 2000s to take on major new functionality as needs became apparent, ranging from sharing objects across applications to linking directly with the Internet or accommodating Web services. In general, Microsoft projects, like XP projects, can re-architect within limits to accommodate new functionality. However, it also appears that Microsoft teams

⁴Michael A. Cusumano and David B. Yoffie, *Competing on Internet Time: Lessons from Netscape and Its Battle with Microsoft*, Free Press/Simon and Schuster, New York, 1998, p. 250.

have learned to plan more than they did in the early days and, unlike XP projects, generally have product release plans that stretch one or two versions into the future.

Pair Programming. Perhaps the most unusual technique used in XP projects is for team members to write code in pairs—that is, with two people sitting at one computer. The idea is that one person is always reviewing what the other person is writing to catch potential errors as well as to spread knowledge of how the product works at a detailed level. This practice may seem to cut engineering productivity in half, but some research suggests that, because team members more quickly catch errors by working in pairs, and errors can become extremely time-consuming and costly to fix later in a project, programming productivity may actually increase.⁵ There is no rule at Microsoft for pair programming, though occasionally some people have experimented with this technique. However, there are analogies. One is the Microsoft practice of pairing each programmer with a “tester buddy” who continually writes test cases for the programmer’s code. Another less formal practice is for programmers to have “buddies” that read their code, instead of more intensive code inspections by larger groups in formal meetings.

Collective Product (Code)

⁵See Alistair Cockburn and Laurie Williams, “The Costs and Benefits of Pair Programming,” unpublished conference paper, available at http://collaboration.csc.ncsu.edu/laurie/Papers/XP_Sardinia.PDF.

Ownership. To reduce bottlenecks in making changes to a product, the general rule is that any XP team member can change any part in the system in any module. Team members take individual responsibility for tasks and delivery dates, but they share ownership of the actual code—the computer instructions that make software work. Microsoft and most other software companies I know assign individual ownership to code modules to make sure someone is responsible for each part of the product. However, in companies like Microsoft where code lives on for many years, code does become collectively owned over time, by default.

Continuous Integration. XP team members synchronize their changes and merge or “integrate” them into the evolving code base at least daily as well as run their tests before and after the integration exercise. If any problems occur, they must fix the code immediately. In this way, the evolving product is always supposed to work. This idea of daily builds and integration testing is also fundamental to Microsoft-style development, used regularly at the company since approximately 1989 along with the practice of breaking projects into shorter “milestone” subprojects of a few weeks or months duration. Daily builds became more difficult to do with massive systems like Windows, and Microsoft has found ways to rely more on branch or subsystem builds. It is also true that, for large projects at Microsoft, the major responsibili-

ties for integration have shifted to dedicated build teams, though individual programmers still run quick tests themselves before checking into the project build. Microsoft’s build tests may not have the same intensity or effectiveness as XP-style continuous integration, but the basic goal of continuous integration remains an important similarity between XP and Microsoft-style iterative development.

Minimal Overtime. There is a social or psychological element to XP practices in that Beck and other advocates strongly discourage programmers from working overtime. Overtime may be inevitable in critical phases of a project, such as just before the final ship date. But the basic philosophy is that projects requiring large amounts of overtime generally reflect poor scheduling and bad management, and place unnecessary stress on the whole team. The XP ideal, as Beck says, is mutual respect and collaboration among programmers. In contrast, at Microsoft and most other software companies, we often see marketing goals take priority over professional goals in a project, forcing companies to use overtime to compensate for unrealistic schedules or poor project management and teamwork. Another approach common in Windows and other Microsoft groups, rather than simply relying on overtime, is to delay products, sometimes for years.

On-site Customer Representative. XP projects do not want to design a project for some abstract

GOOD PROGRAMMERS GENERALLY GRAVITATE TOWARD SIMILAR GOOD ENGINEERING PRACTICES THAT MAKE SENSE FOR THEIR CONTEXTS.

set of users or a distant customer as envisioned by an engineer or marketing manager. Again, the goal is to avoid mistakes and wasted time. XP teams try to work alongside the customer for a custom project or with representatives of the customer, such as product managers, for more packaged or standardized products. Though developers and program managers drive designs in Microsoft, it is standard practice to have product managers and program managers represent the customer in the design process, and to use usability labs and beta releases to get feedback on designs.

Design (Coding) Standards. XP team members are supposed to use the same design notations or standards. The goal here is to facilitate communication and collective code ownership, that is, to make it possible for any team member to understand and change any part of the product or system, during a particular project or in the future. It has been common in Microsoft's application projects to use a standard coding nomenclature and is frequently used in operating system development as well.

In conclusion, every XP practice except simple design and perhaps collective code ownership has some analogues in a

Microsoft-style development process. This suggests about an 80% overlap. Beck argues that many of these similarities are superficial, and that is no doubt true. But there is also a reason for the similarities, superficial or not—good programmers generally gravitate toward similar good engineering practices that make sense for their contexts. I think Microsoft more closely resembled XP in its earlier years. Today, it would benefit from internalizing some XP concepts more deeply, such as the importance of simple design and more direct involvement of customer representatives.

But why choose an iterative development style to begin with? Usually, the reason is the need to evolve or change designs more easily during a project without paying too much of a penalty such as through increased bugs. In fact, research by myself and colleagues on Hewlett-Packard's software development practices indicated the benefits of a more controlled waterfall-style process disappeared in comparison to iterative projects when (and only when) projects used a combination of iterative techniques and good basic practices. These prac-

tices were: early prototypes shown to customers (to reduce the need for late changes), continuous integration and regression testing (to make sure working functions did not break when engineers added new functionality), and design reviews (again, to ensure the project was doing the right thing before building too much of the product).

Of course, XP also has inspired critics. I hear most often that these techniques seem appropriate only for small-scale projects with top-notch programmers who can adapt to working in pairs. XP also presents problems for contract work that requires early detailed proposals and cost estimates, and detailed documentation. These criticisms are valid, though I believe iterative development practices are useful for any software project where the upfront specifications are incomplete—which is nearly always the case. We can also break large projects into smaller projects—if the system architecture is sufficiently modular and the integration builds actually work. ■

MICHAEL CUSUMANO (cusumano@mit.edu) is a professor at the MIT Sloan School of Management and author of *The Business of Software*, Free Press/Simon and Schuster, 2004.

⁶See Alan MacCormack et al., "Trade-offs between Productivity and Quality in Selecting Software Development Practices," *IEEE Software* 20, 5 (Sept./Oct. 2003), 78–85.